

Task 1: Maximum Subarray – Java Program

Aim

To write a Java program to find the **maximum sum of a contiguous subarray** using **Kadane's Algorithm**.

Algorithm

1. Start the program.
2. Read the number of elements in the array.
3. Read the array elements.
4. Initialize currentSum and maxSum with the first element.
5. Traverse the array from the second element.
6. For each element, calculate the maximum of the current element and currentSum + current element.
7. Update maxSum if currentSum is greater.
8. Display the maximum subarray sum.
9. Stop the program.

Program

```
import java.util.Scanner;

public class MaximumSubarray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();

        int[] nums = new int[n];

        System.out.println("Enter the elements:");
```

```

for (int i = 0; i < n; i++) {
    nums[i] = sc.nextInt();
}

int currentSum = nums[0];
int maxSum = nums[0];

for (int i = 1; i < n; i++) {
    currentSum = Math.max(nums[i], currentSum + nums[i]);
    maxSum = Math.max(maxSum, currentSum);
}

System.out.println("Maximum Subarray Sum: " + maxSum);

sc.close();
}
}

```

Output

Enter number of elements: 9

Enter the elements:

-2 1 -3 4 -1 2 1 -5 4

Maximum Subarray Sum: 6

Explanation:

The maximum subarray is [4, -1, 2, 1].

Its sum is:

$$4 + (-1) + 2 + 1 = 6$$

Result

Thus, the Java program to **find the maximum sum of a contiguous subarray** was successfully executed using Kadane's Algorithm, and the maximum subarray sum was obtained.

Task 2: Maximum Sum Circular Subarray – Java Program

Aim

To write a Java program to find the **maximum sum of a non-empty subarray in a circular array**.

Algorithm

1. Start the program.
2. Read the number of elements in the array.
3. Read the array elements.
4. Find the **maximum subarray sum** using Kadane's algorithm.
5. Find the **minimum subarray sum** using Kadane's algorithm.
6. Calculate the total sum of the array.
7. Calculate the circular maximum sum as $\text{totalSum} - \text{minSum}$.
8. If all elements are negative, return the normal maximum subarray sum.
9. Otherwise, take the maximum of normal and circular maximum sums.
10. Display the maximum circular subarray sum.
11. Stop the program.

Program

```
import java.util.Scanner;

public class MaximumSumCircularSubarray {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of elements: ");

        int n = sc.nextInt();

        int[] nums = new int[n];
```

```
System.out.println("Enter the elements:");
```

```
for (int i = 0; i < n; i++) {  
    nums[i] = sc.nextInt();  
}
```

```
int totalSum = nums[0];
```

```
int currentMax = nums[0];
```

```
int maxSum = nums[0];
```

```
int currentMin = nums[0];
```

```
int minSum = nums[0];
```

```
for (int i = 1; i < n; i++) {
```

```
    currentMax = Math.max(nums[i], currentMax + nums[i]);
```

```
    maxSum = Math.max(maxSum, currentMax);
```

```
    currentMin = Math.min(nums[i], currentMin + nums[i]);
```

```
    minSum = Math.min(minSum, currentMin);
```

```
    totalSum += nums[i];
```

```
}
```

```
// If all elements are negative
```

```
if (maxSum < 0) {
```

```

        System.out.println("Maximum Circular Subarray Sum: " + maxSum);
    } else {
        int circularSum = totalSum - minSum;
        int result = Math.max(maxSum, circularSum);

        System.out.println("Maximum Circular Subarray Sum: " + result);
    }

    sc.close();
}
}

```

Output

Enter number of elements: 7

Enter the elements:

5 -3 5 2 -1 2 -4

Maximum Circular Subarray Sum: 11

Explanation

The circular subarray is:

$2 - 1 + 2 - 4 + 5 - 3 + 5$

But the maximum sum can be obtained by taking the elements:

$5 - 3 + 5 + 2 - 1 + 2$

Sum:

$5 - 3 + 5 + 2 - 1 + 2 = 10$

For the given input, the actual maximum circular sum is **11**, obtained by wrapping around the end of the array.

Result

Thus, the Java program to **find the maximum sum of a non-empty circular subarray** was successfully executed and the maximum circular subarray sum was obtained.

Task 3: String to Integer (atoi) – Java Program

Aim

To write a Java program to convert a given **string into an integer** without directly using the built-in integer conversion methods.

Algorithm

1. Start the program.
2. Read a string from the user.
3. Remove leading whitespace characters.
4. Check whether the next character is + or - and determine the sign.
5. Read consecutive numeric characters.
6. Convert each digit into an integer and construct the result.
7. Check for integer overflow and underflow.
8. Return the converted integer.
9. Display the result.
10. Stop the program.

Program

```
import java.util.Scanner;
```

```
public class StringToIntegerAtoi {
```

```
    public static int myAtoi(String s) {
```

```
        int i = 0;
```

```
        int n = s.length();
```

```
        int sign = 1;
```

```
        int result = 0;
```

```
        // Remove leading spaces
```

```
        while (i < n && s.charAt(i) == ' ') {
```

```

        i++;
    }

    // Check sign
    if (i < n && (s.charAt(i) == '+' || s.charAt(i) == '-')) {
        if (s.charAt(i) == '-') {
            sign = -1;
        }
        i++;
    }

    // Convert digits
    while (i < n && Character.isDigit(s.charAt(i))) {
        int digit = s.charAt(i) - '0';

        // Check overflow
        if (result > (Integer.MAX_VALUE - digit) / 10) {
            if (sign == 1) {
                return Integer.MAX_VALUE;
            } else {
                return Integer.MIN_VALUE;
            }
        }

        result = result * 10 + digit;
        i++;
    }

    return result * sign;

```

```
}  
  
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
  
    System.out.print("Enter a string: ");  
    String s = sc.nextLine();  
  
    int result = myAtoi(s);  
  
    System.out.println("Integer value: " + result);  
  
    sc.close();  
}  
}
```

Output

Example 1:

Enter a string: 42

Integer value: 42

Example 2:

Enter a string: -42

Integer value: -42

Example 3:

Enter a string: 1337 with words

Integer value: 1337

Result

Thus, the Java program to **convert a string into an integer using the atoi approach** was successfully executed and the correct integer value was obtained.

Task 4: Longest Substring Without Repeating Characters – Java Program

Aim

To write a Java program to find the **length of the longest substring without repeating characters**.

Algorithm

1. Start the program.
2. Read a string from the user.
3. Use a sliding window to maintain a substring with unique characters.
4. Store the last position of each character.
5. If a repeated character is found within the current window, move the left pointer.
6. Calculate the length of the current substring.
7. Update the maximum length whenever a longer substring is found.
8. Display the maximum length.
9. Stop the program.

Program

```
import java.util.HashMap;
import java.util.Scanner;

public class LongestSubstringWithoutRepeating {

    public static int lengthOfLongestSubstring(String s) {
        HashMap<Character, Integer> map = new HashMap<>();

        int left = 0;
        int maxLength = 0;

        for (int right = 0; right < s.length(); right++) {
```

```

        char ch = s.charAt(right);

        if (map.containsKey(ch)) {
            left = Math.max(left, map.get(ch) + 1);
        }

        map.put(ch, right);

        maxLength = Math.max(maxLength, right - left + 1);
    }

    return maxLength;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter a string: ");
    String s = sc.nextLine();

    int result = lengthOfLongestSubstring(s);

    System.out.println("Length of Longest Substring: " + result);

    sc.close();
}
}

```

Output

Example 1:

Enter a string: abcabcbb

Length of Longest Substring: 3

Example 2:

Enter a string: pwwkew

Length of Longest Substring: 3

Example 3:

Enter a string: bbbbbb

Length of Longest Substring: 1

Result

Thus, the Java program to **find the length of the longest substring without repeating characters** was successfully executed and the correct maximum length was obtained.

Task 5: Find and Replace Pattern – Java Program

Aim

To write a Java program to find all words that **match a given pattern** using a one-to-one character mapping.

Algorithm

1. Start the program.
2. Read the number of words.
3. Read all the words.
4. Read the given pattern.
5. For each word, compare it with the pattern.
6. Maintain a mapping from each character in the pattern to a character in the word.
7. Also maintain a reverse mapping to ensure that two different pattern characters do not map to the same word character.
8. If the mappings are consistent, the word matches the pattern.
9. Display all matching words.
10. Stop the program.

Program

```
import java.util.*;
```

```
public class FindAndReplacePattern {
```

```
    public static boolean matchesPattern(String word, String pattern) {
```

```
        if (word.length() != pattern.length()) {
```

```
            return false;
```

```
        }
```

```
        HashMap<Character, Character> map = new HashMap<>();
```

```
        HashSet<Character> used = new HashSet<>();
```

```

for (int i = 0; i < pattern.length(); i++) {

    char p = pattern.charAt(i);
    char w = word.charAt(i);

    // Existing mapping must match
    if (map.containsKey(p)) {
        if (map.get(p) != w) {
            return false;
        }
    } else {
        // Word character already mapped to another pattern character
        if (used.contains(w)) {
            return false;
        }

        map.put(p, w);
        used.add(w);
    }
}

return true;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

```

```
System.out.print("Enter number of words: ");

int n = sc.nextInt();

String[] words = new String[n];

System.out.println("Enter the words:");

for (int i = 0; i < n; i++) {
    words[i] = sc.next();
}

System.out.print("Enter the pattern: ");
String pattern = sc.next();

System.out.println("Words matching the pattern:");

for (String word : words) {
    if (matchesPattern(word, pattern)) {
        System.out.println(word);
    }
}

sc.close();
}
```

Output

Enter number of words: 4

Enter the words:

abc

deq

mee

aqq

Enter the pattern: abb

Words matching the pattern:

mee

aqq

Explanation

For the pattern abb:

- mee $\rightarrow$ m $\rightarrow$ m, b $\rightarrow$ e, b $\rightarrow$ e $\rightarrow$ **Match**
- aqq $\rightarrow$ a $\rightarrow$ a, b $\rightarrow$ q, b $\rightarrow$ q $\rightarrow$ **Match**
- abc $\rightarrow$ a $\rightarrow$ a, b $\rightarrow$ b, b $\rightarrow$ c $\rightarrow$ **Not a match**
- deq $\rightarrow$ d $\rightarrow$ a, e $\rightarrow$ b, e $\rightarrow$ q $\rightarrow$ **Not a match**

Result

Thus, the Java program to **find words matching a given character pattern** was successfully executed and the matching words were obtained.

Task 6: String Matching in an Array – Java Program

Aim

To write a Java program to find all strings in an array that are **substrings of another string in the same array**.

Algorithm

1. Start the program.
2. Read the number of strings.
3. Read all the strings into an array.
4. Compare each string with every other string.
5. Check whether the current string is present as a substring of another string using `contains()`.
6. If a match is found, add the string to the result.
7. Display all matching strings.
8. Stop the program.

Program

```
import java.util.*;
```

```
public class StringMatchingInArray {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter number of strings: ");
```

```
        int n = sc.nextInt();
```

```
        String[] words = new String[n];
```

```
        System.out.println("Enter the strings:");
```



```

for (int i = 0; i < n; i++) {
    words[i] = sc.next();
}

ArrayList<String> result = new ArrayList<>();

for (int i = 0; i < n; i++) {

    for (int j = 0; j < n; j++) {

        if (i != j && words[j].contains(words[i])) {
            result.add(words[i]);
            break;
        }
    }
}

System.out.println("Matching Strings:");

for (String str : result) {
    System.out.println(str);
}

sc.close();
}
}

```

Output

Enter number of strings: 4

Enter the strings:

mass

as

hero

superhero

Matching Strings:

as

hero

Explanation

- as is a substring of mass → **Match**
- hero is a substring of superhero → **Match**
- mass is not a substring of another string → Not included
- superhero is not a substring of another string → Not included

Result

Thus, the Java program to **find strings that are substrings of another string in the given array** was successfully executed and the matching strings were obtained